

Persistência da Informação

Ficheiros

UA.DETI.POO

Introdução

- ❖ Sem capacidade de interagir com o "resto do mundo", o nosso programa torna-se inútil
 - Esta interação designa-se “input/output” (I/O)
- ❖ Problema → Complexidade
 - Diferentes e complexos dispositivos de I/O (ficheiros, consolas, canais de comunicação, ...)
 - Diferentes formatos de acesso (sequencial, aleatório, binário, caracteres, linha, palavras, ...)
- ❖ Necessidade → Abstração
 - Libertar o programador da necessidade de lidar com as especificidade e complexidade de cada I/O

Operações de entrada/saída (I/O)

Entrada

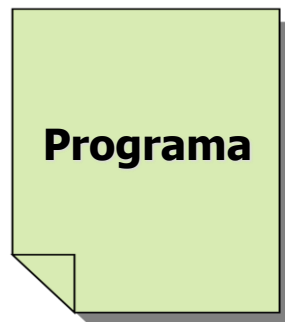
teclado



leitura



Programa



escrita



Saída

monitor



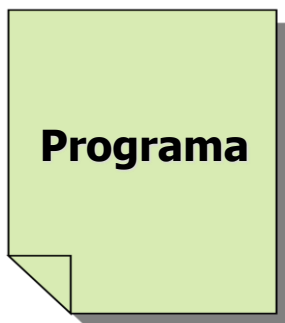
ficheiro



leitura



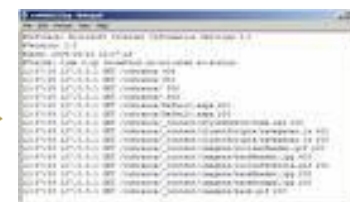
Programa



escrita



ficheiro



Java IO e NIO

- ❖ A linguagem java disponibiliza dois packages para permitir operações de entrada/saída de dados

- ❖ **Java IO**

- Stream oriented
- Blocking IO

- ❖ **Java NIO (new IO)**

- Buffer oriented
- Non blocking IO
- Channels
- Selectors



Ficheiros de Texto

Ficheiros – Classes principais

❖ Java IO

- File
- BufferedReader
- FileReader
- FileWriter

- RandomAccessFile

❖ Java NIO

- Path
- Paths
- Files

- SeekableByteChannel

java.io.File

- ❖ A classe *File* representa quer um nome de um ficheiro quer o conjunto de ficheiros num diretório
- ❖ Fornece informações e operações úteis sobre ficheiros e diretórios
 - `canRead`, `canWrite`, `exists`, `getName`, `isDirectory`, `isFile`, `listFiles`, `mkdir`, ...
- ❖ Exemplos:

```
File file1 = new File("io.txt");
File file2 = new File("C:/tmp/", "io.txt");
File file3 = new File("P00/Slides");

if (!file1.exists()) { /* do something */ }
if (!file3.isDirectory()) { /* do something */ }
```

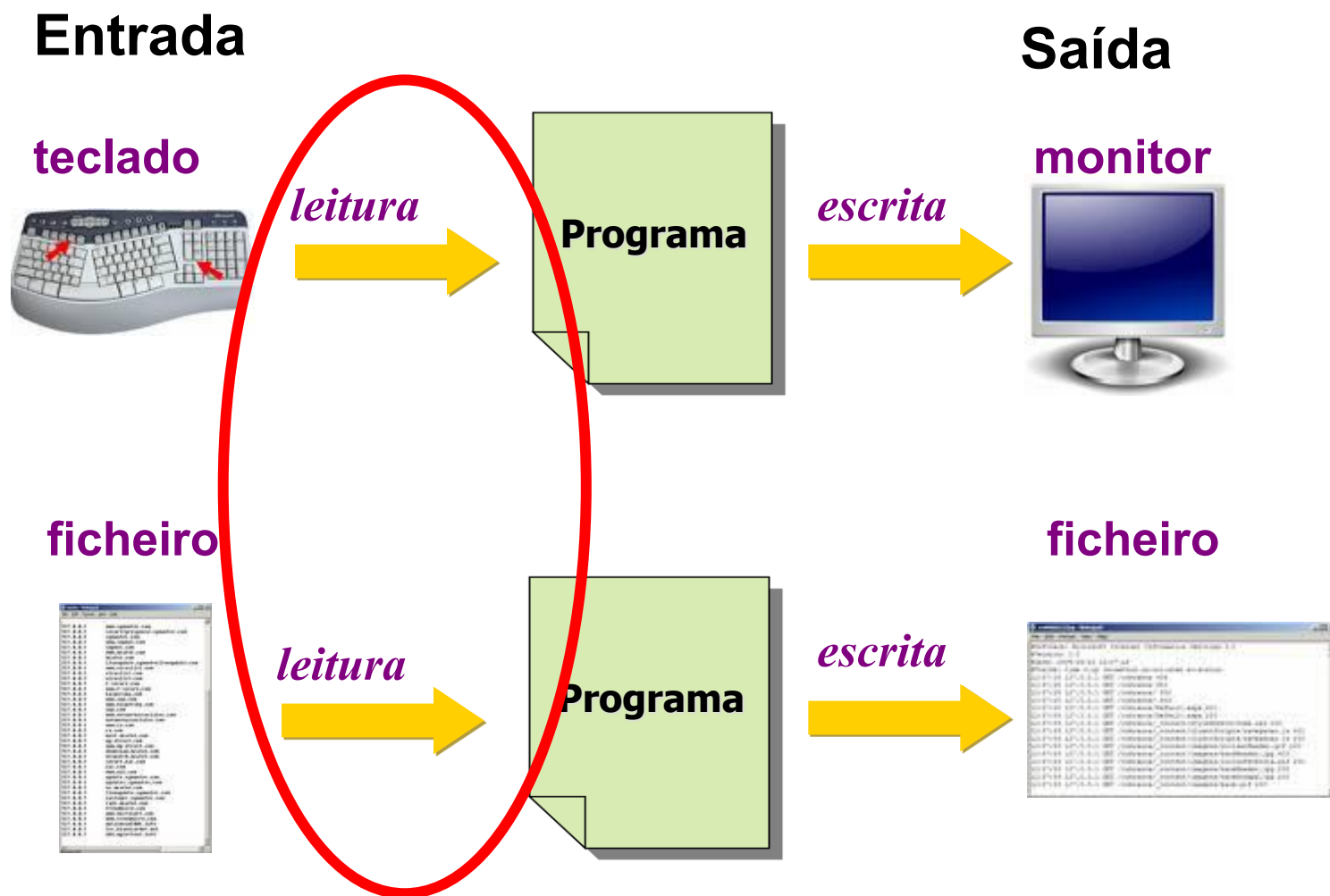
Exemplo – Listar um Diretório

```
import java.io.*;

public class DirList {
    public static void main(String[] args) {
        File directorio = new File("src/");
        File[] arquivos = directorio.listFiles();
        for (File f : arquivos) {
            System.out.println(f.getAbsolutePath());
        }
    }
}
```

```
Com java.nio
Path dir = ...
try (DirectoryStream<Path> stream =
    Files.newDirectoryStream(dir)) {
    for (Path entry: stream) { ... }
}
```

Operações de entrada/saída (I/O)



Ler dados ...

Comparing File Reading Classes

- ❖ Java gives you several ways to read files. Here's when to pick each one:
 - [Scanner](#) - best for **simple text**. It can split text into lines, words, or numbers (e.g., `nextInt()`, `nextLine()`).
 - `BufferedReader` - best for **large text files**. It is faster, uses less memory, and can read full lines with `readLine()`.
 - [FileInputStream](#) - best for **binary files** (like images, PDFs, or audio)

https://www.w3schools.com/java/java_bufferedReader.asp

Java Scanner

Last updated: January 5, 2024



Written by:
Eugen Paraschiv

Java IO

Java Scanner

reference

1. Overview of *Scanner*

In this quick tutorial, we'll illustrate how to use the **Java *Scanner*** class – to read input and find and skip patterns different delimiters.

2. Scan a File

First – let's see how to read a file using *Scanner*.

In the following example – we read a file containing "*Hello world*" into tokens:

```
@Test
public void whenReadFileWithScanner_thenCorrect() throws IOException{
    Scanner scanner = new Scanner(new File("test.txt"));

    assertTrue(scanner.hasNext());
    assertEquals("Hello", scanner.next());
    assertEquals("world", scanner.next());

    scanner.close();
}
```

No <https://www.baeldung.com/java-scanner>

Also, note how we close the scanner when we're done using it.

- 1. Overview of Scanner
- 2. Scan a File
- 3. Convert InputStream to String
- 4. Scanner vs. BufferedReader
- 5. Scan Input From the Console Using New Scanner(System.in)
- 6. Validate Input
- 7. Scan a String
- 8. Find Pattern
- 9. Skip Pattern
- 10. Change Scanner Delimiter
- 11. Handling NoSuchElementException
- 12. nextLine() After nextXXX() Methods
- 13. Conclusion

... usando java.util.Scanner

- ❖ Classe que facilita a leitura de tipos primitivos e de Strings a partir de uma fonte de entrada.

- Ler do teclado

```
Scanner sc1 = new Scanner(System.in);  
int i = sc1.nextInt();
```

- Ler de uma string

```
Scanner sc2 = new Scanner("really long\nString\n\t\tthat I want to pick  
apart\n");  
while (sc2.hasNextLine())  
    System.out.println(sc2.nextLine());
```

- Ler de um ficheiro

```
Scanner input = new Scanner(new File("words.txt"));  
while (input.hasNextLine())  
    System.out.println(input.nextLine());
```

Java obriga a identificar problemas

```
package aula01;
```

```
import java.util.Scanner;
```

```
import java.io.File;
```

```
public class Ficheiro {
```

Run | Debug

```
public static void m
```

```
    Scanner input = new Scanner(new File(pathname:"words.txt"));
```

```
    while (input.hasNextLine())
```

```
    |     System.out.println(input.nextLine());
```

```
    input.close();
```

```
}
```

```
}
```

Unhandled exception type FileNotFoundException Java(16777384)

View Problem (Alt+F8) Quick Fix... (Ctrl+.) Fix using Copilot (Ctrl+I)

Java obriga a identificar problemas

```
package aula01;
```

```
import java.util.Scanner;
```

```
import java.io.File;
```

```
public class
```

```
test > src > aula01 > Ficheiro.java > Ficheiro.java
1 package aula01;
2
3 import java.util.Scanner;
4 import java.io.File;
5
6
7 public class Ficheiro {
8
9     Run | Debug
10     public static void main() {
11         Scanner input = new Scanner(new File("word.txt"));
12         while (input.hasNextLine()) {
13             System.out.println(input.nextLine());
14         }
15     }
16 }
17 }
```

Unhandled exception type FileNotFoundException [16777284]

java.io.File.File(String pathname)

Creates a new File instance by converting the given pathname string into an abstract pathname. If the given string is the empty string, then the result is the empty abstract pathname.

• Parameters:

- **pathname** A pathname string

• Throws:

- **NullPointerException** - If the path

View Problem (Alt+F8) Quick Fix... (Ctrl+.)

VSCoode dá pistas do problema e propõe soluções

Quick Fix

- 💡 Add throws declaration
- 💡 Surround with try-with-resources
- 💡 Surround with try/catch
- 🌟 Fix using Copilot
- 🌟 Explain using Copilot

<https://www.geeksforgeeks.org/handle-an-ioexception-in-java/>

Java obriga a identificar problemas

```
package aula01;

import java.util.Scanner;
import java.io.File;
import java.io.FileNotFoundException;

public class Ficheiro {
    Run | Debug
    public static void main(String[] args) {
        try (Scanner input = new Scanner(new File(pathname:"words.txt"))) {
            while (input.hasNextLine())
                System.out.println(input.nextLine());
            input.close();
        } catch (FileNotFoundException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
    }
}
```

... através de exceções: opções

❖ Necessário declarar que pode gerar exceção

```
public class TestReadFile {  
    public static void main(String[] args) throws FileNotFoundException {  
        Scanner input = new Scanner(new File("words.txt"));  
        while (input.hasNextLine())  
            System.out.println(input.nextLine());  
    }  
}
```

❖ Ou lidar com exceção (matéria de outra aula)

```
public class TestReadFile {  
    public static void main(String[] args) {  
        try {  
            Scanner input = new Scanner(new File("words.txt"));  
            while (input.hasNextLine())  
                System.out.println(input.nextLine());  
            input.close();  
        } catch (FileNotFoundException e) {  
            System.out.println("Ficheiro não existente!");  
        }  
    }  
}
```

Problemas: passar para a frente (mais à frente vemos como lidar com Exceções)

❖ Exemplo 1: sem tratamento de exceções

```
public class TestReadFile
{
    public static void main(String[] args) throws FileNotFoundException
    {
        Scanner input = new Scanner(new File("words.txt"));
        while (input.hasNextLine())
            System.out.println(input.nextLine());
    }
}
```

❖ O ficheiro "words.txt" deve estar:

- Na pasta local, se o programa for executado através de linha de commando
- Na pasta do projeto, caso seja executado a partir do IDE

java.nio – Leitura de ficheiros de texto

❖ Podemos usar métodos estáticos das classes **Files** e **Paths** do package `java.nio.file`.

❖ Exemplo 4:

```
public class ReadFileIntoList {  
    public static void main(String[] args) throws IOException {  
        List<String> lines = Files.readAllLines(Paths.get("words.txt"));  
        for (String ln : lines)  
            System.out.println(ln);  
    }  
}
```

Guide to BufferedReader

Last updated: January 8, 2024



Written by:
baeldung



Reviewed by:
Josh Cummings

Java IO

Java FileReader

1. Overview

BufferedReader is a class which simplifies reading text from a character input stream. It buffers the character order to enable efficient reading of text data.

In this tutorial, we're going to look at how to use the *BufferedReader* class.

- 1. Overview
- 2. When to Use BufferedReader
 - 2.1. Buffering Another Reader
 - 2.2. Buffering a Stream
 - 2.3. BufferedReader vs Scanner**
- 3. Reading Text With BufferedReader
- 4. Other Useful Methods
- 5. Conclusion

2. When to Use *BufferedReader*

In general, *BufferedReader* comes in handy if we want to read text from any kind of input source whether that be files, sockets, or something else.

Simply put, **it enables us to minimize the number of I/O operations by reading chunks of characters and storing them in an internal buffer.** While the buffer has data, the reader will read from it instead of directly from the

or <https://www.baeldung.com/java-buffered-reader>

<https://docs.oracle.com/javase/8/docs/api/java/io/BufferedReader.html>

2.1. Buffering Another Reader

Guide to BufferedReader

Last updated: January 8, 2024



Written by:
baeldung



Reviewed by:
Josh Cummings

Java IO

Java FileReader

- 1. Overview
- 2. When to Use BufferedReader

1. Overview

BufferedReader is a class which simplifies reading in order to enable efficient reading of text data. In this tutorial, we're going to look at how to use it.

2. When to Use *BufferedReader*

In general, *BufferedReader* comes in handy in situations where you are reading from files, sockets, or something else.

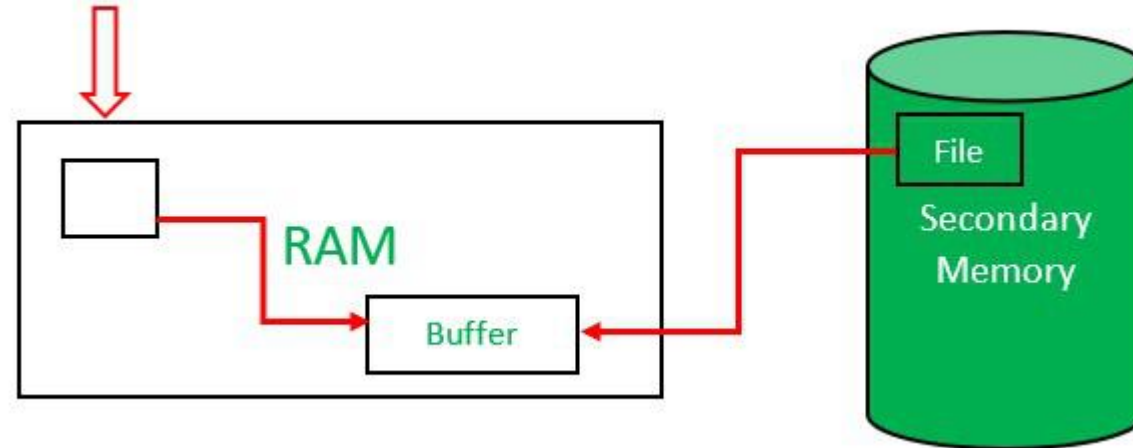
Simply put, **it enables us to minimize the number of I/O operations by reading chunks of characters and storing them in an internal buffer.** While the buffer has data, the reader will read from it instead of directly from the source.

For more information, see <https://www.baeldung.com/java-buffered-reader>

or <https://docs.oracle.com/javase/8/docs/api/java/io/BufferedReader.html>

2.1. Buffering Another Reader

Buffered Reader object



Reader

s Scanner
fferedReader
;

```

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class ReadTSVBuffered {

    public static void main(String[] args) {
        try (BufferedReader reader = new BufferedReader(new FileReader("students.tsv"))) {

            reader.readLine(); // Skip header line

            String line;
            while ((line = reader.readLine()) != null) {
                String[] parts = line.split("\t");
                String name = parts[0];
                int g1 = Integer.parseInt(parts[1]);
                int g2 = Integer.parseInt(parts[2]);
                int g3 = Integer.parseInt(parts[3]);

                double average = (g1 + g2 + g3) / 3.0;
                System.out.println("Name: " + name + ", Average: " + average);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

```

import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class ReadTSVScanner {

    public static void main(String[] args) {

        try (Scanner scanner = new Scanner(new File("students.tsv"))) {

            scanner.useDelimiter("\\t|\\n");

            // Skip header (4 tokens: Name, Grade1, Grade2, Grade3)
            scanner.next(); scanner.next(); scanner.next(); scanner.next();

            while (scanner.hasNext()) {
                String name = scanner.next();
                int g1 = scanner.nextInt();
                int g2 = scanner.nextInt();
                int g3 = scanner.nextInt();

                double average = (g1 + g2 + g3) / 3.0;
                System.out.println("Name: " + name + ", Average: " + average);
            }
        } catch (FileNotFoundException e) {
            e.printStackTrace();
        }
    }
}

```

Name	Grade1	Grade2	Grade3
John	85	90	78
Mary	92	88	95
Alex	70	75	80

```

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class ReadTSVBuffered {

    public static void main(String[] args) {
        try (BufferedReader reader = new BufferedReader(new FileReader("students.tsv"))) {

            reader.readLine(); // Skip header line

            String line;
            while ((line = reader.readLine()) != null) {
                String[] parts = line.split("\t");
                String name = parts[0];
                int g1 = Integer.parseInt(parts[1]);
                int g2 = Integer.parseInt(parts[2]);
                int g3 = Integer.parseInt(parts[3]);

                double average = (g1 + g2 + g3) / 3.0;
                System.out.println("Name: " + name + ", Average: " + average);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

```

import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class ReadTSVScanner {

    public static void main(String[] args) {

        try (Scanner scanner = new Scanner(new File("students.tsv"))) {

            scanner.useDelimiter("\\t|\\n");

            // Skip header (4 tokens: Name, Grade1, Grade2, Grade3)
            scanner.next(); scanner.next(); scanner.next(); scanner.next();

            while (scanner.hasNext()) {
                String name = scanner.next();
                int g1 = scanner.nextInt();
                int g2 = scanner.nextInt();
                int g3 = scanner.nextInt();

                double average = (g1 + g2 + g3) / 3.0;
                System.out.println("Name: " + name + ", Average: " + average);
            }
        } catch (FileNotFoundException e) {
            e.printStackTrace();
        }
    }
}

```

Name	Grade1	Grade2	Grade3
John	85	90	78
Mary	92	88	95
Alex	70	75	80

2.3. BufferedReader vs Scanner

As an alternative, we could use the *Scanner* class to achieve the same functionality as with *BufferedReader*.

However, there are significant differences between these two classes which can make them either more or less convenient for us, depending on our use case:

- *BufferedReader* is synchronized (thread-safe) while *Scanner* is not
- *Scanner* can parse primitive types and strings using regular expressions
- *BufferedReader* allows for changing the size of the buffer while *Scanner* has a fixed buffer size
- *BufferedReader* has a larger default buffer size
- *Scanner* hides *IOException*, while *BufferedReader* forces us to handle it
- *BufferedReader* is usually faster than *Scanner* because it only reads the data without parsing it

With these in mind, **if we are parsing individual tokens in a file, then *Scanner* will feel a bit more natural than *BufferedReader*. But, just reading a line at a time is where *BufferedReader* shines.**

If needed, we also have a [guide on *Scanner*](#) as well.



2.3. BufferedReader vs Scanner

As an alternative, we could use the *Scanner* class to achieve the same functionality as with *BufferedReader*.

However, there are significant differences between these two classes which can make them either more or less convenient for us, depending on our use case:

- *BufferedReader* is synchronized (thread-safe) while *Scanner* is not
- *Scanner* can parse primitive types and strings using regular expressions
- *BufferedReader* allows for changing the size of the buffer while *Scanner* has a fixed buffer size
- *BufferedReader* has a larger default buffer size
- *Scanner* hides *IOException*, while *BufferedReader* forces us to handle it
- *BufferedReader* is usually faster than *Scanner* because it only reads the data without parsing it

With these in mind, **if we are parsing individual tokens in a file, then *Scanner* will feel a bit more natural than *BufferedReader*. But, just reading a line at a time is where *BufferedReader* shines.**

If needed, we also have a [guide on *Scanner*](#) as well.



E no mundo real? Alguns exemplos

```

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class ReadFileExample {

    public static void main(String[] args) {

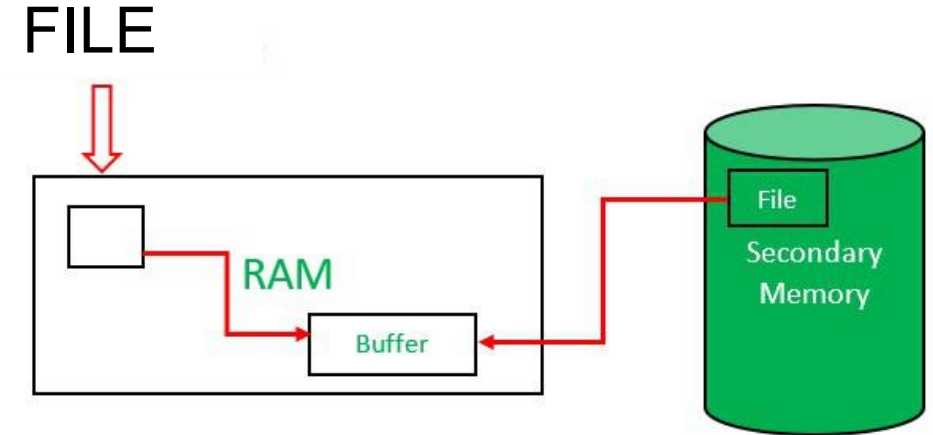
        try {
            BufferedReader reader = new BufferedReader(new FileReader("c:\\poo\\test.py"));

            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }

            reader.close();

        } catch (IOException e) {
            System.out.println("Error reading file: " + e.getMessage());
        }
    }
}

```

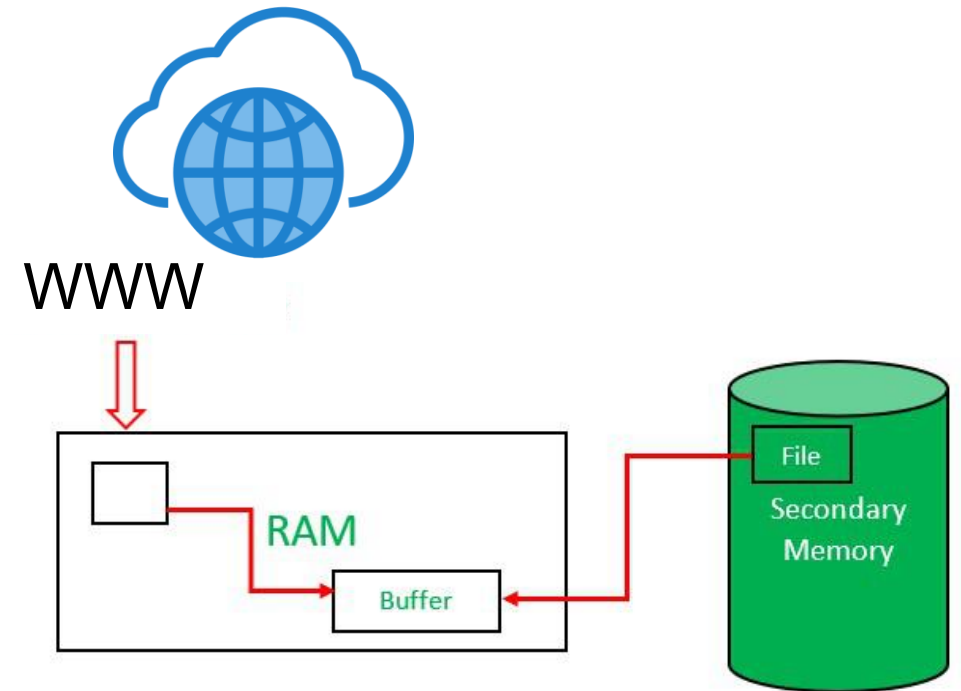


```

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;
import java.net.URL;
import java.net.URLConnection;

public class WebTest {
    public static void main(String[] args) {
        String urlString = "http://www.ua.pt"; // Change to your desired URL
        try {
            URL url = new URL(urlString);
            URLConnection connection = url.openConnection();
            BufferedReader reader = new BufferedReader(
                new InputStreamReader(connection.getInputStream()));
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
            reader.close();
        } catch (IOException e) {
            System.out.println("Error reading from URL: " + e.getMessage());
            e.printStackTrace();
        }
    }
}

```



```

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class WeatherExample {

    public static void main(String[] args) {
        try {
            String url = "https://api.open-meteo.com/v1/forecast"
                + "?latitude=38.72" + "&longitude=-9.13"
                + "&current_weather=true";

            HttpClient client = HttpClient.newHttpClient();

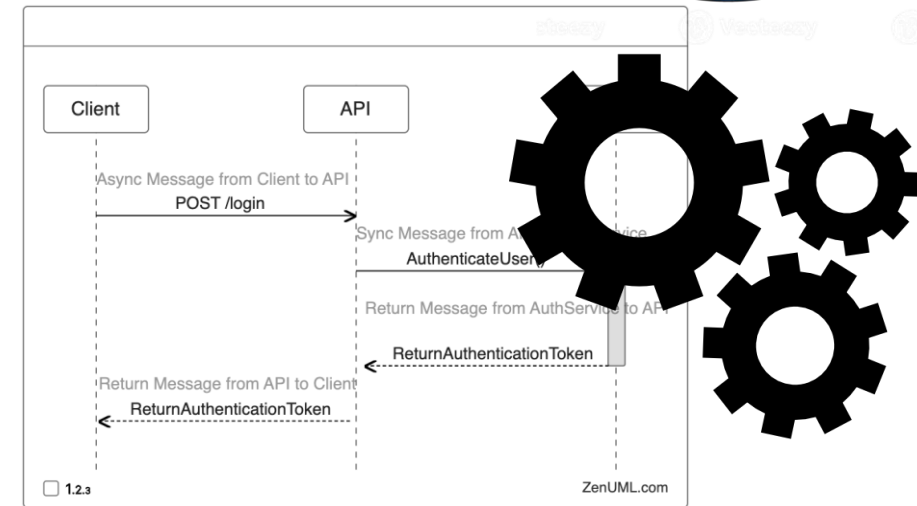
            HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(url))
                .GET()
                .build();

            HttpResponse<String> response =
                client.send(request, HttpResponse.BodyHandlers.ofString());

            System.out.println("Raw JSON response:");
            System.out.println(response.body());

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```



Escrita de dados

Operações de entrada/saída (I/O)

Entrada

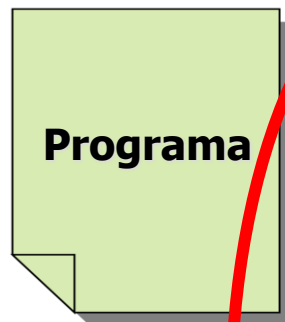
teclado



leitura



Programa



escrita



Saída

monitor



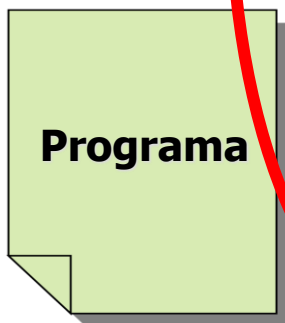
ficheiro



leitura



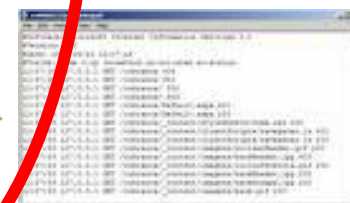
Programa



escrita



ficheiro



Escrita de ficheiros de texto

❖ classe java.io.PrintWriter

- Permite-nos usar os métodos println e printf para escrever em ficheiros de texto.
- Formata os valores de tipos primitivos em texto, tal como quando impressos no écran.

```
public class FileWritingDemo {  
    public static void main(String[] args) throws IOException {  
        PrintWriter out = new PrintWriter(new File("file1.txt"));  
        out.println("Fim de semana na praia");  
        out.printf("Viagem: %d\nHotel: %d\n", 345, 1000);  
        out.close();  
    }  
}
```

Escrita de ficheiros de texto – append

- ❖ Podemos acrescentar (append) mais informação a um ficheiro existente

```
public class FileWritingDemo {  
    public static void main(String[] args) throws IOException {  
        FileWriter fileWriter = new FileWriter("file1.txt", true);  
        PrintWriter printWriter = new PrintWriter(fileWriter);  
        printWriter.append("a acrescentar mais umas notas...\n");  
        printWriter.close();  
    }  
}
```

Sumário

- ❖ java.io e java.nio
- ❖ Representar ficheiros e directórios com **File**
- ❖ Ler ficheiros de texto com **Scanner**, **BufferedReader**
- ❖ Escrever ficheiros de texto com **PrintWriter**
- ❖ Muitas outras classes existem para manipular I/O
 - <https://docs.oracle.com/javase/tutorial/essential/io/>

The Java™ Tutorials

[Hide TOC](#)

Basic I/O

[I/O Streams](#)
[Byte Streams](#)
[Character Streams](#)
[Buffered Streams](#)
[Scanning and](#)
[Formatting](#)
[Scanning](#)
[Formatting](#)
[I/O from the Command](#)
[Line](#)
[Data Streams](#)
[Object Streams](#)
[File I/O \(Featuring NIO.2\)](#)
[What Is a Path? \(And](#)
[Other File System](#)
[Facts\)](#)
[The Path Class](#)
[Path Operations](#)
[File Operations](#)
[Checking a File or](#)
[Directory](#)
[Deleting a File or](#)
[Directory](#)
[Copying a File or](#)
[Directory](#)
[Moving a File or](#)
[Directory](#)
[« Previous](#) • [Trail](#) • [Next »](#)
[Home Page](#) > [Essential Java Classes](#)

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.

See [Dev.java](#) for updated tutorials taking advantage of the latest releases.

See [Java Language Changes](#) for a summary of updated language features in Java SE 9 and subsequent releases.

See [JDK Release Notes](#) for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Lesson: Basic I/O

This lesson covers the Java platform classes used for basic I/O. It first focuses on *I/O Streams*, a powerful concept that greatly simplifies I/O operations. The lesson also looks at serialization, which lets a program write whole objects out to streams and read them back again. Then the lesson looks at file I/O and file system operations, including random access files.

Most of the classes covered in the `I/O Streams` section are in the `java.io` package. Most of the classes covered in the `File I/O` section are in the `java.nio.file` package.

I/O Streams

- [Byte Streams](#) transfer raw bytes of data.

- [Character Streams](#) transfer text.

- [Buffered Streams](#) optimize input and output by reducing the number of calls to the native I/O methods.

<https://docs.oracle.com/javase/tutorial/essential/io/>

Basic I/O

I/O Streams

Byte Streams

Character Streams

Buffered Streams

Scanning and

Formatting

Scanning

Formatting

I/O from the Command

Line

Data Streams

Object Streams

File I/O (Featuring NIO.2)

What Is a Path? (And

Other File System

Facts)

The Path Class

Path Operations

File Operations

Checking a File or

Directory

Deleting a File or

Directory

Copying a File or

Directory

Moving a File or

Directory

« Previous • Trail • Next »

Home Page > Essential Java Classes

I/O Streams

- **Byte Streams** handle I/O of raw binary data.
- **Character Streams** handle I/O of character data, automatically handling translation to and from the local character set.
- **Buffered Streams** optimize input and output by reducing the number of calls to the native API.
- **Scanning and Formatting** allows a program to read and write formatted text.
- **I/O from the Command Line** describes the Standard Streams and the Console object.
- **Data Streams** handle binary I/O of primitive data type and `String` values.
- **Object Streams** handle binary I/O of objects.

I/O Streams

- **Byte Streams** handle I/O of raw binary data.

- **Character Streams**

for

- **Buffered Streams** optimize input and output by reducing the number of calls to the native

<https://docs.oracle.com/javase/tutorial/essential/io/>